

Practical No. 9

1. Title

Development of a Spring MVC Application using Dependency Injection

2. Aim

To develop a web application using the Spring MVC framework by implementing Dependency Injection for better modularity, loose coupling, and efficient component management.

3. Learning Objectives

- Understand Spring MVC architecture
- Learn Dependency Injection (DI)
- Understand Controller, Model, View roles
- Learn IoC (Inversion of Control) container
- Develop dynamic web applications
- Configure Spring using annotations/XML
- Understand request handling and data binding

4. Software Requirements

- Framework: Spring MVC
- Build Tool: Maven
- IDE: Eclipse / IntelliJ
- Server: Apache Tomcat
- JDK: 8+
- Browser: Chrome

5. Theory Background (Clean Version)

Spring MVC

Spring MVC is a framework based on the **Model-View-Controller (MVC)** pattern used to build web applications.

MVC Architecture

- **Model** → Handles data & business logic
- **View** → JSP/HTML UI
- **Controller** → Handles user requests

6. Assignment Questions

9.1. Payment Processing System In a modern e-commerce platform, customers are provided with multiple payment options such as credit card, debit card, and UPI. The system should be designed in such a way that the payment method can be selected and changed dynamically without modifying the core business logic of the application. Develop a Spring-based application that uses Dependency Injection to integrate different payment services. The application should define a common interface for payment processing and implement multiple payment methods. The main service class should use the injected payment service to process transactions, and switching between payment methods should be done through configuration without changing the Java code.

Input:

PaymentService.java

```
package com.example.payment;
```

```
public interface PaymentService {  
    void pay(double amount);  
}
```

CreditCardPayment.java

```
package com.example.payment;
```

```
public class CreditCardPayment implements PaymentService {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using Credit Card");  
    }  
}
```

DebitCardPayment.java

```
package com.example.payment;
```

```
public class DebitCardPayment implements PaymentService {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using Debit Card");  
    }  
}
```

UPIPayment.java

```
package com.example.payment;
```

```
public class UPIPayment implements PaymentService {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using UPI");  
    }  
}
```

PaymentProcessor

```
package com.example.payment;
```

```
public class PaymentProcessor {  
  
    private PaymentService paymentService;  
  
    public void setPaymentService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
  
    public void processPayment(double amount) {  
        paymentService.pay(amount);  
    }  
}
```

MainApp.java

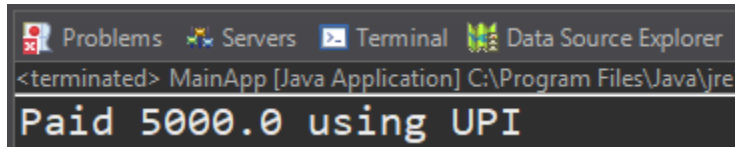
```
package com.example.payment;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class MainApp {  
    public static void main(String[] args) {  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("beans.xml");  
  
        PaymentProcessor processor =  
            (PaymentProcessor) context.getBean("paymentProcessor");  
  
        processor.processPayment(5000);  
    }  
}
```

Bean.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <!-- Change this class to switch payment -->  
    <bean id="paymentService" class="com.example.payment.UPIPayment"/>  
  
    <bean id="paymentProcessor" class="com.example.payment.PaymentProcessor">  
        <property name="paymentService" ref="paymentService"/>  
    </bean>
```

</beans>

Output:



9.2.Notification System In many real-world applications such as banking systems, e-commerce platforms, and social media applications, users receive notifications through different channels like email, SMS, or mobile alerts. The system should be flexible enough to support multiple notification methods and allow easy switching between them without altering the main application logic. Design a Spring-based application that uses Dependency Injection to send notifications through different services. Create a common notification interface and multiple implementations such as email and SMS. The main service class should rely on the injected notification service, and the type of notification should be configurable externally using Spring configuration.

Input:

EmailService.java

```
package com.example.notification;
```

```
public class EmailService implements NotificationService {  
    @Override  
    public void sendNotification(String message) {  
        System.out.println("Email Sent: " + message);  
    }  
}
```

NotificationManager.java

```
package com.example.notification;
```

```
public class NotificationManager {  
    private NotificationService notificationService;  
  
    public void setNotificationService(NotificationService notificationService) {  
        this.notificationService = notificationService;  
    }  
  
    public void notifyUser(String message) {  
        notificationService.sendNotification(message);  
    }  
}
```

NotificationService.java

```
package com.example.notification;
```

```
public interface NotificationService {  
    void sendNotification(String message);  
}
```

SMSService.java

```
package com.example.notification;
```

```
public class SMSService implements NotificationService {  
    @Override  
    public void sendNotification(String message) {  
        System.out.println("SMS Sent: " + message);  
    }  
}
```

Test.java

```
package com.example.notification;
```

```
78
```

```
import org.springframework.context.ApplicationContext;
```

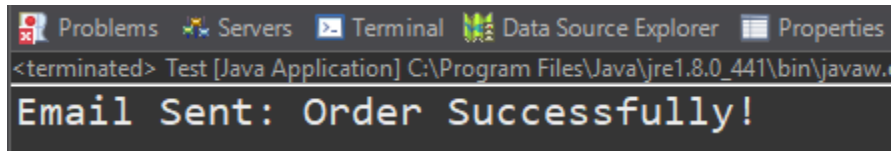
```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
public class Test {  
    public static void main(String[] args) {  
        ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");  
        NotificationManager manager = (NotificationManager) context.getBean("notificationManager");  
        manager.notifyUser("Order Successfully!");  
    }  
}
```

applicationContext.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
        http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans.xsd">  
    <bean id="notificationService" class="com.example.notification.EmailService"/>  
    <bean id="notificationManager" class="com.example.notification.NotificationManager">  
        <property name="notificationService" ref="notificationService"/>  
    </bean>  
</beans>
```

Output:



7. Conclusion:

The experiment successfully demonstrates the use of Dependency Injection in Spring to integrate multiple payment services. It ensures loose coupling, flexibility, and easy switching of payment methods, which is essential for modern scalable applications.